

Karpathy +HTML 繁中精解

Karpathy：把 LLM 回應結構化為 HTML（繁體中文精解版）

來源：[@karpathy 推文 2053872850101285137](#) 譯註版本：v1.0 · 2026-05-19 · vins-hub/vinshub
授權：以 CC BY-SA 4.0 釋出

一句話濃縮

在你的任何 LLM prompt 結尾，加上一句「Format your entire response as a complete HTML document」，然後把輸出存成 response.html 用瀏覽器打開。

就這樣。一行指令，把 LLM 從「會寫文字的工具」變成「會寫文件的工具」。

為什麼這條看似簡單的提示這麼重要

⚠️ 2026-05-19 自我驗證更新：先前版本引用了「Stone Age」這句話作為 Karpathy 原話，但實際上那是 [quasa.io](#) 的 paraphrase。詳細比對：[SELF-VALIDATION.md](#)

Karpathy 的真正論述：Audio → Input, Vision → Output

“audio is the human-preferred input to AIs but vision (images/animations/video) is the preferred output from them. Around a ~third of our brains are a massively parallel processor dedicated to vision, it is the 10-lane superhighway of information into brain.”

—— Karpathy 原推文（2026-05-12）

人類對 AI 的 I/O 偏好是不對稱的：

方向	人類偏好	為什麼
Input → AI（你給 AI）	Audio（語音）	講話比打字快，思考流暢
AI → Output（AI 給你）	Vision（視覺：圖像 / 動畫 / 影片）	大腦 1/3 是視覺處理器，10 線道資訊高速公路

HTML 不是孤立技巧，是「vision-as-AI-output」這個大論述在 2026 的具體實踐。

4 級演進階梯（Karpathy 原文）

- 1) raw text (難讀、費力)
- 2) markdown (現行 default)
- 3) HTML ← 我們在這裡
- ↓ ...
- n) interactive neural videos / simulations
(由 diffusion neural net 直接生成的互動影片)

“the extrapolation (though the technology doesn’t exist just yet) ends in some kind of interactive videos generated directly by a diffusion neural net.”

HTML 不是終點，是第一步。在 Neuralink-esque BCI 之前，這是目前能做的最大跳躍。

為何 HTML 是當前最佳選擇

- 排版、顏色、字型、間距全套樣式系統
- Tables、lists、accordions、details/summary 可摺疊區塊
- 暗色模式自動切換 (prefers-color-scheme)
- 響應式 layout
- 連 SVG / Canvas / 簡易互動 (<details>、onclick) 都不用加 build step

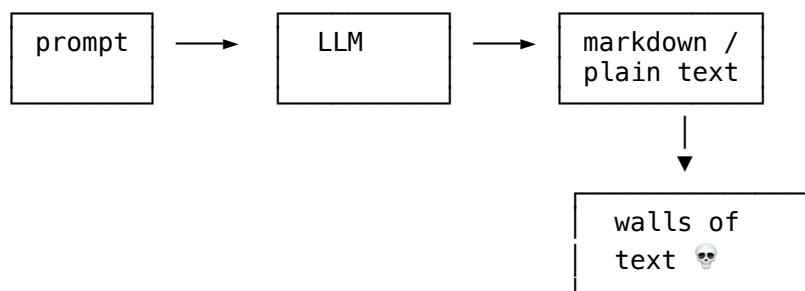
你早就有了瀏覽器——這是地球上裝機率最高的軟體——卻把它閒置，繼續看 markdown 純文字。

致謝 Thariq：方法的原始長文

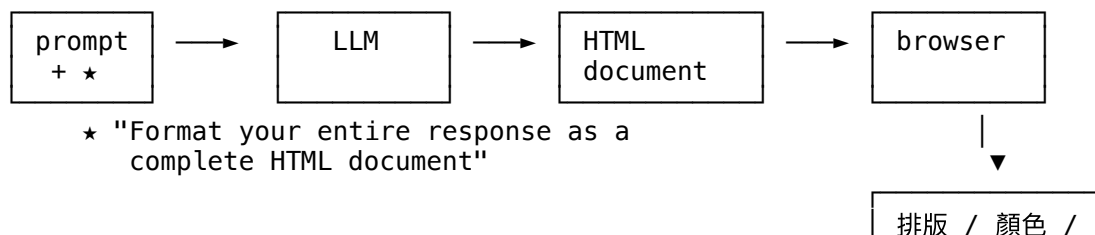
Karpathy 這條推文實際上是 **quote tweet** 回應 [@trq212 Thariq 在 May 9 寫的文章](#)「Using Claude Code: The Unreasonable Effectiveness of HTML」。Thariq (Anthropic Claude Code team) 才是最早系統性論述「**markdown** 是 **agent** 與人類溝通的 **dominant format**，但 **HTML** 可以做得更好」的人。Karpathy 推廣了這個方法 + 補上 vision-as-output 的理論基礎。

心智模型

傳統工作流：



Karpathy 工作流：



學習路徑

章節	內容	適合誰
01. 輸出端瓶頸	為何純文字 / markdown 已經不夠	所有 LLM 使用者
02. HTML 方法論	完整 prompt 範本 + 工作流 SOP	想立刻上手的人
03. 七大應用場景	Plan / Report / Wiki / Dashboard / Tutorial / Cheatsheet / Code Review	看你的 use case 對號入座
04. Prompt 模板庫	9 個可複製貼上的高品質模板	立刻可用
05. 反模式	別這樣做的 8 個陷阱	避免踩雷
06. 進階：互動、可下載、分享	從靜態 HTML 升級到 mini app	想極致化的人
07. 跟 12-factor agents 的關聯	為何這條跟 Context Engineering 同源	體系派
Appendix: HTML CSS 速查	即查即用的常用 pattern	工具書

Karpathy 的 TLDR 金句

「The input/output mind meld between humans and AIs is ongoing and there is a lot of work to do and significant progress to be made, way before jumping all the way into neuralink-esque BCIs and all that. For what’s worth exploring at the current stage, hot tip try ask for HTML.」

「人機 I/O 心靈融合是個持續過程，在跳到 Neuralink-style BCI 之前還有很多進步空間。就目前可以探索的階段，hot tip：試試請 AI 給你 HTML。」

順帶一提，Karpathy 也提到 **slideshow** 是另一個有效的 trick（同樣 prompt 結尾要求 LLM 用 slideshow 形式呈現）。這份指南聚焦在 HTML，但 slideshow 是延伸應用。

30 秒上手

打開你的 ChatGPT / Claude / Gemini / 任何 LLM，貼這段：

[你原本的問題]

- Format your entire response as a complete HTML document with:
- Clean modern typography (system fonts)
 - Color-coded sections by importance
 - Collapsible <details> for less-critical content
 - Tables for any structured comparison
 - Dark/light mode support via prefers-color-scheme

– Responsive layout (max-width: 820px, centered)

Output only the HTML, no markdown wrapper.

把輸出存成 `response.html`，雙擊打開。你會在第一次看到的瞬間就理解這個方法的價值。

為什麼這個方法被低估

絕大多數人接觸 LLM 都是透過 ChatGPT-like 的 chat UI。在 chat UI 裡：

- HTML 被 sandbox 在 iframe 或被禁止
- 樣式有 chat 框架硬規定
- 沒辦法存檔、分享、版本控制

於是大家默認「LLM 的輸出是 chat message」。但 LLM 本身並不知道這件事——你叫它輸出 HTML，它就輸出 HTML。

Karpathy 提示的洞察：把 LLM 從「chat 互動」拉回「文件生成器」的本質。文件天然就是 HTML。

主要受益場景

用戶	受益
研究 / 學習	LLM 整理的知識點可摺疊、可搜尋、可重看
產品 / 規劃	給 LLM 一個 spec，它直接吐出可分享的 plan deck
資料分析	表格、圖表、條件分析可互動展開
教學文件	階段式揭露、code 區塊高亮、範例可摺疊
報告寫作	直接得到可印 / 可分享的 HTML 報告
個人 wiki / 第二大腦	每次查詢都生成可保存的 HTML 卡片

譯註者的補充

這個方法在中文社群討論度遠低於它的實際效益。Karpathy 用 5 行推文講完的事，抵得上 **100 個 prompt engineering 課程** 的玄學。

它之所以神奇，不是因為 **HTML** 多複雜，而是因為它讓你有意識地拒絕「**LLM 輸出 = chat message**」這個預設。一旦你意識到 LLM 可以為你生成完整文件，你會發現純 markdown 是個非常窄的天花板。

接下來

 [Chapter 01: 輸出端瓶頸 — 為何純文字已經不夠](#)

第 00 章：Karpathy 原推文完整對照

來源：[@karpathy/status/2053872850101285137](https://twitter.com/karpathy/status/2053872850101285137) · 2026-05-12 · 2.8M views 抓取方式：
gstack/browse headless browser 實際載入

英文原文（完整）

Andrej Karpathy @karpathy

This works really well btw, at the end of your query ask your LLM to “structure your response as HTML”, then view the generated file in your browser. I’ve also had some success asking the LLM to present its output as slideshows, etc.

More generally, imo audio is the human-preferred input to AIs but vision (images/animations/video) is the preferred output from them. Around a ~third of our brains are a massively parallel processor dedicated to vision, it is the 10-lane superhighway of information into brain. As AI improves, I think we’ll see a progression that takes advantage:

1. raw text (hard/effortful to read)
2. markdown (bold, italic, headings, tables, a bit easier on the eyes) <– current default
3. HTML (still procedural with underlying code, but a lot more flexibility on the graphics, layout, even interactivity) <– early but forming new good default ...4,5,6,...
- n. interactive neural videos/simulations

Imo the extrapolation (though the technology doesn’t exist just yet) ends in some kind of interactive videos generated directly by a diffusion neural net. Many open questions as to how exact/procedural “Software 1.0” artifacts (e.g. interactive simulations) may be woven together with neural artifacts (diffusion grids), but generally something in the direction of the recently viral [@zan2434/status/2046982383430496444](https://twitter.com/zan2434/status/2046982383430496444)

There are also improvements necessary and pending at the input. Audio nor text nor video alone are not enough, e.g. I feel a need to point/gesture to things on the screen, similar to all the things you would do with a person physically next to you and your computer screen.

TLDR The input/output mind meld between humans and AIs is ongoing and there is a lot of work to do and significant progress to be made, way before jumping all the way into neuralink-esque BCIs and all that. For what’s worth exploring at the current stage, hot tip try ask for HTML.

Quote tweet 對象：

Thariq @trq212 · May 9

Article: Using Claude Code: The Unreasonable Effectiveness of HTML

Markdown has become the dominant file format used by agents to communicate with us. It’s simple, portable, has some rich text capability and is easy for you to edit. Claude has even gotten...

繁體中文逐段翻譯

Andrej Karpathy (@karpathy)

順道一提，這招效果非常好——在你 query 結尾請 LLM 「把你的回應結構化為 **HTML**」，然後在瀏覽器打開生成的檔案。我也試過請 LLM 用簡報 (**slideshow**) 等形式呈現輸出，也滿成功的。

從更廣的角度看，個人觀點：**audio** 是人類偏好的「給 AI」輸入方式，但 **vision** (圖像 / 動畫 / 影片) 是人類偏好的「AI 給人」輸出方式。我們大腦約 1/3 是專門處理視覺的大規模平行處理器，它是進入大腦的「10 線道資訊高速公路」。AI 變強後，我預期會看到一條善用這個事實的演進路徑：

1) **raw text** (純文字，難讀、費力) 2) **markdown** (粗體、斜體、標題、表格——對眼睛較友善) ← 目前 **default** 3) **HTML** (底下仍是程式碼，但圖形、layout、甚至互動的彈性大很多) ← 新興、正成為新 **default** ...4, 5, 6, ... n) 互動式 **neural** 影片 / 模擬

我認為極端外推 (雖然技術現在還不存在) 會結束在某種由 **diffusion neural net** 直接生成的互動式影片。在「精確/程序化的 Software 1.0 artifact (例如互動式模擬) 如何與 neural artifact (diffusion grid) 編織在一起」這個問題上有很多 open question，但大致方向是像 [@zan2434](#) 最近爆紅的那個方向。

輸入端也有改善需要做。Audio 或 text 或 video 任一單獨都不夠——例如我覺得需要指 / 比劃螢幕上的東西，類似於你跟一個實際坐你旁邊看你螢幕的人會做的所有動作。

TLDR：人類跟 AI 之間的「**input/output** 心靈融合」是個持續演進的議題，在跳到 **Neuralink-esque BCI** 之前還有大量工作要做、大量進步空間。就目前可以探索的階段，**hot tip**：試試請 AI 給你 **HTML**。

上下文：被引用的 Thariq 原始文章

Karpathy 的推文是 quote tweet 形式，被引用的是 Thariq (@trq212, Anthropic Claude Code team) 在 5/9 發表的 X Article：

標題：「Using Claude Code: The Unreasonable Effectiveness of HTML」

開頭段落 (X 預覽可見)：> Markdown has become the dominant file format used by agents to communicate with us. It's simple, portable, has some rich text capability and is easy for you to edit. Claude has even gotten...

這篇是「LLM 輸出 **HTML**」方法的原始系統論述。Karpathy 推廣它並加上 vision-as-output 的理論基礎。

要看 Thariq 原文，需要 X premium 或從 Thariq 的個人 site 查找。

從原推文可以拆解出的精確論點

#	論點	章節對應
1	方法：query 結尾要求 LLM 輸出 HTML	Ch 02 方法論
2	延伸：slideshow 也是類似 trick	Ch 02 / Ch 04
3	理論：audio = preferred input, vision = preferred output	Ch 01 為何有效
4	科學：人腦 1/3 是視覺處理器（10 線道）	Ch 01
5	演進階梯：text → markdown → HTML → ... → neural video	Ch 01 / README
6	未來：diffusion neural net 直接生成互動影片	Ch 07
7	input 端也要進步：gesture/point at screen	延伸思考
8	TLDR ：BCI 之前用 HTML 探索	README

引文準確性聲明

本 repo 第一版（commit 13ac436）部分引用「Output is still stuck in the Stone Age」是 **quasa.io** 的 **paraphrase**，不是 **Karpathy** 原話。已在 [SELF-VALIDATION.md](#) 詳細說明並修正。

如果你看到本 repo 任何地方引用 Karpathy 時用了引號 "...”，請對照本章原文確認。本章的英文版才是 source of truth。

接下來

[➡ Chapter 01: 輸出端瓶頸 — 為何純文字已經不夠](#)

[← 回 README](#)

第 01 章：輸出端瓶頸 — 為何純文字已經不夠

核心句：LLM 的輸入端跑在 21 世紀（multimodal、視覺、語音、影片），輸出端還停在 1991 年的純文字 email。這個 input/output 維度錯配是當前 AI 應用最大的隱形浪費。

對稱性破缺：input vs output

過去 3 年，LLM 的「輸入端」進化驚人：

年份	輸入升維
2023	純文字 prompt
2024	圖像 input (GPT-4V / Claude 3 Vision)
2024	文件 input (PDF / Word 直接上傳)
2025	即時語音 input (Realtime API)
2025	影片 input (Gemini 1.5+)
2026	Multimodal interleaved (圖文音影同 prompt)

而輸出端呢？

年份	輸出狀態
2023	純文字
2024	純文字 / markdown
2025	純文字 / markdown
2026	純文字 / markdown (還是！)

Karpathy 點破：「**Output is still stuck in the Stone Age — mostly plain text or lightly formatted Markdown.**」

3 年來，input 端做了 5 次跳躍，output 端幾乎原地踏步。

為何 markdown 不夠

Markdown 是 2004 年 John Gruber 設計的，目標是「易寫的 **HTML** 替代品」。它的設計約束：

- 1. 純文字編輯器友善 (不需 toolbar)
- 2. 轉成 **HTML** 簡單
- 3. 語法簡潔

這些約束在 2004 是優點，在 **2026** 是限制：

Markdown 能做 **Markdown 不能做 (但 HTML 可以)**

粗體、斜體	顏色、字型、字距
標題層級	卡片、邊框、陰影
列表	摺疊區塊、tab 切換
表格 (簡單)	表格樣式、sticky header、scroll
Code block	Syntax highlighting (要 renderer 配合)
連結	Button、hover、tooltip
圖片	SVG inline、canvas、互動圖表

Markdown 能做 Markdown 不能做（但 HTML 可以）

- 暗色模式自動切換
- 響應式設計（mobile / desktop）
- 動畫、過渡、micro-interaction
- 直接列印 / 存 PDF（瀏覽器原生）

Markdown 是無格式的「思考骨架」；HTML 是有格式的「呈現作品」。它們不該被混為一談。

「Walls of Text」的代價

當 LLM 回答你一個複雜問題，給的是一大段純文字（或 markdown），你的大腦要做什麼？

1. 斷句：把連續文字切成意義塊
2. 重排優先級：哪些是核心結論、哪些是支撐論據
3. 建立心智地圖：各段之間的關係
4. 記住結構：等等下捲時要記得上面講了什麼

這四件事，本來應該是「呈現層」做的，現在全推給你的工作記憶。對複雜問題，這就是為什麼讀 LLM 回答時你會覺得「累」——你的大腦在做 layout engine 的事。

Karpathy 用詞：**Walls of Text**（文字之牆）——讀完一整堵牆，沒人能記得最上方寫了什麼。

HTML 把工作搬回呈現層

當 LLM 直接吐 HTML：

```
<!doctype html>
<html>
<head>
  <style>
    body { max-width: 820px; margin: 2rem auto; font-family: system-ui; line-height: 1.6; }
    .conclusion { background: #f0f9ff; border-left: 4px solid #0369a1; padding: 1rem; }
    .secondary { color: #666; font-size: 0.9em; }
    details { margin: 0.5em 0; }
    @media (prefers-color-scheme: dark) {
      body { background: #1a1a1a; color: #ddd; }
      .conclusion { background: #1e3a5f; }
    }
  </style>
</head>
<body>
  <div class="conclusion">
    <strong>結論</strong>：因為 X，所以你該選 A 不選 B。
  </div>

  <details>
    <summary>展開：X 是什麼？</summary>
```

```
<p class="secondary">X 是 ...</p>
</details>

<details>
  <summary>展開：為什麼不是 B ?</summary>
  <p class="secondary">B 的問題在於 ...</p>
</details>
</body>
</html>
```

讀者體驗瞬間升級：

- 結論優先：藍色邊框 box 第一眼就抓住
- 次要細節摺疊：想看再展開，不想看不擋路
- 暗色模式：晚上看眼睛不痛
- 可印 / 可存：瀏覽器 Cmd+P 就 PDF

這些都是 LLM 內部「早就知道」的呈現直覺，只是你沒問它。

為什麼大家都沒這樣做

三個原因：

1. Chat UI 鎖死預設

ChatGPT / Claude.ai / Gemini 等 chat UI 把 LLM 輸出強制 **sandbox** 在 **chat bubble**。你會看到：- HTML 不渲染（或在 iframe 裡限制）- 樣式被 chat 框架覆寫 - 沒有“save as html”按鈕

於是大家從來沒看過 LLM 輸出完整 HTML 的樣子。

2. Prompt engineering 課程沒教

絕大多數 prompt engineering 教學講 chain-of-thought、few-shot、role prompting...沒人講「換個輸出格式」這件事。因為它不在「prompt 技巧」傳統範疇——它是「輸出格式」決策。

3. Markdown 已被當預設

人類已經習慣 markdown 在 GitHub、Notion、Obsidian。Markdown 「夠好用」掩蓋了「它不是極限」的事實。

一個 mental 實驗

想像兩個版本的同一份產品 spec：

Version A

12,000 字 markdown，no formatting

你會花 30 分鐘讀完，記得 30%

Version B

同樣內容，輸出為 HTML：核心結論在頂、需求區可摺疊、表格 sticky header、TOC sidebar

你會花 10 分鐘讀完，記得 70%，且可以隨時回來查

B 的內容沒有比 A 多——只是「呈現層」做了它的工作。

從 Karpathy 的更廣脈絡看

Karpathy 不只在講 HTML，他在講輸出形式的演化。他的長期預測：

“Interactive videos, 3D simulations, or dynamic visualizations generated on the fly. Instead of reading about something, you’ll experience it.”

「互動影片、3D 模擬、即時生成的動態視覺化。你不是讀關於某件事，而是體驗它。」

HTML 不是終點，是第一步。從 plain text → markdown → HTML → interactive simulation → multimodal experience，是一條清楚的演化路徑。今天能做到的最大跳躍，就是 **markdown → HTML**。

本章小結

觀念

Input 升維 vs Output 停滯

Markdown 是 2004 年產物

Walls of Text 推工作給讀者大腦

HTML 把工作搬回呈現層

Chat UI / 教學體系造成盲點

為什麼重要

找到當前 LLM 應用的最大低效點

它的設計約束不該綁住 2026 的 LLM 輸出

呈現層該做的事不該推給工作記憶

LLM 本來就會這事，只是你沒問

大家從沒看過 LLM 完整 HTML 輸出長什麼樣

接下來

 [Chapter 02: HTML 方法論 — 完整 prompt 範本 + workflow SOP](#)

理解「為何」之後，下一步是「如何」：到底要怎麼寫 prompt、怎麼操作流程，才能穩定拿到高品質 HTML 輸出。

[← 回 README](#) · [← Ch 01](#)

第 02 章：HTML 方法論 — 完整 Prompt 範本 + 工作流 SOP

核心句：把 LLM 從「chat 助手」切換成「HTML 文件產生器」，只需要一個 prompt 結尾的指令。但要穩定拿到高品質結果，需要 4 個層次的精細化：基礎指令 → 樣式約束 → 結構提示 → 互動元素。

4 層 Prompt 精細化階梯

Level 1：最基礎（一行版）

Format your entire response as a complete HTML document.

結果：能跑、但樣式 LLM 自由發揮，品質浮動。

Level 2：基礎 + 樣式約束（推薦起點）

Format your entire response as a complete HTML document with:

- Clean modern typography using system fonts
- Max-width 820px, centered
- Line-height 1.65, comfortable reading
- Dark/light mode support via prefers-color-scheme
- Headings with subtle border-bottom
- Code blocks with monospace font and gray background

Output only the HTML, no markdown wrapper, no explanation.

結果：穩定的中高品質輸出，可以開始當文件用。

Level 3：基礎 + 樣式 + 結構提示

Format your entire response as a complete HTML document.

STYLE REQUIREMENTS:

- System fonts: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif
- Max-width: 820px, margin: 2rem auto, padding: 0 1.5rem
- Line-height: 1.65, color: #222 (light) / #ddd (dark)
- Headings: border-bottom 1px solid, padding-bottom 0.3em
- Code: Menlo/Consolas/monospace, background #f5f5f5, border-radius 4px
- Tables: full width, alternate row backgrounds
- Dark mode via @media (prefers-color-scheme: dark)

STRUCTURE REQUIREMENTS:

- Start with a "Conclusion" box (colored background, prominent)
- Then "Key Points" as a list with emoji or icons
- Then a comparison table if comparing things
- Then detailed sections, each in a <details> block
- End with "Next Steps" or "Action Items"

OUTPUT:

- Only the complete HTML (with <!doctype html>)
- No markdown code fences around the HTML
- No explanation before or after

結果：可以當交付品用，達到 production-grade 文件質量。

Level 4：完整版 + 互動元素

Format your entire response as a complete, self-contained HTML document.

STYLE: [same as Level 3]

STRUCTURE: [same as Level 3]

INTERACTIVITY:

- Use <details>/<summary> for collapsible sections
- Add a sticky top navigation with anchor links to each section
- For tables with > 5 rows, add a search input that filters rows via inline JS
- For code blocks, add a "Copy" button via inline JS
- Add a table of contents that highlights current section on scroll (intersection observer)
- Make all links open in new tab (target="_blank")

ACCESSIBILITY:

- Semantic HTML (<article>, <section>, <nav>)
- ARIA labels on interactive elements
- Keyboard-navigable (tabindex)
- Min font size 16px

OUTPUT:

- Only the complete HTML, no markdown wrapper
- All CSS inline in <style>
- All JS inline in <script>
- Self-contained: no external dependencies

結果：mini single-page app，可以直接 deploy / 分享。

工作流 SOP

簡單一次性查詢

1. 想問題（無論是什麼問題）
2. 在 prompt 結尾貼上 Level 2 模板
3. 把 LLM 輸出複製到 response.html
4. 瀏覽器打開

時間成本：**+5 秒**。閱讀體驗：**+200%**。

重複型 / 重要文件

1. 把你的 Level 3 / Level 4 模板存成 snippet (macOS 用 Raycast / Alfred)
2. 想生成什麼文件，先寫具體問題
3. 觸發 snippet 自動 append 模板
4. 送 LLM

5. 用 Chrome DevTools 微調 → 把改動丟回 LLM 說「保留這些 style，重新生成內容」
6. 反覆迭代

Pipeline 化（自動化生成報告）

自動化範例 (Python + Anthropic SDK)

```
import anthropic

HTML_TEMPLATE_INSTRUCTION = """
Format your entire response as a complete HTML document with...
(your Level 3 / Level 4 spec here)
"""

def llm_to_html(user_prompt: str, output_path: str) -> None:
    client = anthropic.Anthropic()
    msg = client.messages.create(
        model="claude-opus-4-7",
        max_tokens=8000,
        messages=[{
            "role": "user",
            "content": f"{user_prompt}\n\n---\n\n{HTML_TEMPLATE_INSTRUCTION}"
        }]
    )
    html = msg.content[0].text
    # LLM 可能還是會包 ```html ... ``` 把它剝掉
    html = html.strip().removeprefix("```html").removesuffix("```").strip()
    with open(output_path, "w") as f:
        f.write(html)
    print(f"✓ Saved to {output_path}")
```

呼叫：

```
llm_to_html(
    "Analyze my Q1 sales data: ... [paste data]",
    "/tmp/q1-analysis.html"
)
import webbrowser; webbrowser.open("/tmp/q1-analysis.html")
```

5 行程式碼，把 LLM 變成你自己的文件產生器。

5 個常見問題

Q1：LLM 還是會包 html markdown 怎麼辦？

A：在 prompt 加：

IMPORTANT: Do NOT wrap the HTML in markdown code fences.
Start your response with <!doctype html> directly.

或者程式碼裡用 `strip().removeprefix("```html").removesuffix("```")` 處理。

Q2：輸出截斷了怎麼辦？

A：兩個解法：

1. 提高 **max_tokens**（很多人忘記設這個，預設可能只 1024）
2. 分段生成：先讓 LLM 生成 outline，再分章節各別生成 HTML 片段，最後手動合併

Q3：圖表怎麼辦？

A：兩個方案：

1. **SVG inline**：LLM 直接生成 `<svg>...</svg>`，簡單的 bar / line chart 都可以
2. **Chart.js / D3 via CDN**：把 CDN URL 加到 head，LLM 生成 `<canvas>` + 初始化 JS

範例 prompt 加：

```
For data visualization, use inline <svg> for simple charts (bar, line, pie).  
For complex charts, use Chart.js via CDN:  
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

Q4：要不要加 Tailwind？

A：通常不用。Tailwind 沒在 LLM 訓練資料裡 first-class，且需要 build step / CDN bundle。直接寫 **inline CSS** 反而更穩——LLM 對 raw CSS 駕輕就熟。

Q5：怎麼確保跨 LLM provider 結果一致？

A：不能完全一致。但 Level 3 / Level 4 的具體約束已經把 80% 的變動鎖住。剩下 20% 是各家 LLM 風格差異（Claude 偏 minimal，GPT-4 偏 colorful，Gemini 偏 structured）。挑一家當主力，把模板針對它調 fine-tune。

進階技巧：把 HTML 當成「LLM ↔ 人類」契約

當 LLM 為你輸出 HTML，這個 HTML 不只是「給人讀」，它還是 **LLM** 自我表達的高保真載體。具體表現：

表現	含意
LLM 把某段放進「結論」box	它認為這段最重要
LLM 把某段放進 <code><details></code>	它認為這段是次要細節
LLM 用紅色 background 標記某段	它認為這段是警告 / 風險
LLM 把某些項目做成 sticky	它認為這些是需要持續參考的

讀 **LLM** 寫的 **HTML**，等於讀 **LLM** 對「資訊優先級」的判斷。比讀 markdown 更有資訊量。

範本：你可以立刻用的 5 個模板

我們會在 [第 04 章](#) 給 9 個完整模板。先給你 1 個搶先用：

「任何技術問題分析」模板

[你的技術問題]

Format your entire response as a complete HTML document analyzing this problem.

STRUCTURE:

1. Top: Executive Summary in a colored box (TL;DR in 3 sentences)
2. "Problem Statement" section
3. "Root Cause Analysis" with collapsible details for each possible cause
4. "Recommended Solution" with step-by-step plan
5. "Trade-offs" as a comparison table
6. "Implementation Checklist" as a checklist
7. "Risks & Mitigations"

STYLE:

- System fonts, max-width 820px, centered
- Dark mode via prefers-color-scheme
- Code blocks with syntax highlighting class names
- Tables with subtle borders, alternate row colors

OUTPUT: Complete self-contained HTML, no markdown wrapper.

試一次：拿一個你最近在思考的技術決策，貼進去，看結果。

本章小結

觀念	用途
4 層精細化階梯	從入門到 production-grade
Snippet 化模板	5 秒重複觸發
Pipeline 自動化	把 LLM 變成文件 generator
處理截斷 / 圖表 / Tailwind 等實務問題	邊角案例不卡住
HTML 是 LLM 表達優先級的高保真載體 讀 HTML 比讀 markdown 更有資訊	

接下來

 [Chapter 03: 七大應用場景](#)

知道怎麼寫 prompt 後，下一步是知道什麼情境最值得用這個方法。

[← 回 README](#) · [← Ch 02](#)

第 03 章：七大應用場景

核心句：HTML 輸出不是萬靈丹，但有 7 個場景受益遠遠超過平均——讀者要重複查、要結構化、要比較、要可分享。這 7 個場景應該變成你的「默認觸發條件」。

觸發判斷

什麼時候該 +HTML？兩個條件任一成立就值得：

1. 這個輸出我會看超過 1 次（會回來查、會分享）
2. 這個輸出有結構（多個 section、表格、比較、分階段）

如果是「問一個事實，看完就忘」（例如「Python 的 enumerate 怎麼用？」），純 markdown 就夠。

7 大高 ROI 場景

場景 1：Plan / 規劃文件

為何適合 HTML：plan 通常有層級結構（目標 → 階段 → 任務）、有時程、有 owner、有 dependency。HTML 的 nested details + sticky TOC + table 完美匹配。

Prompt 範本：

Generate a detailed implementation plan for: [你的目標]

Format as a complete HTML document with:

- Top: Goal + Success Criteria in a highlighted box
- "Phases" section with 3-5 phases, each in a <details> block (default open for Phase 1, collapsed others)
- Each phase contains: tasks table (Task / Owner / Estimate / Dependencies / Status)
- "Risks" section with risk table (Risk / Impact / Mitigation)
- "Timeline" as a visual ASCII Gantt or simple SVG
- Sticky TOC sidebar with current section highlight

Style: system fonts, max-width 1100px (wider for tables), dark mode support.

Output: complete HTML, no markdown wrapper.

典型受益：sprint plan、產品 spec、migration plan、deploy runbook。

場景 2：Report / 分析報告

為何適合 HTML：報告要有 executive summary + 詳細分析 + 數據表 + 結論。讀者通常先看 summary，再選擇性深入。<details> 就是為這個發明的。

Prompt 範本：

Analyze the following [數據 / 情況]: [paste data]

Produce a complete HTML report with:

1. Executive Summary at top (highlighted box, 3 sentences max)
2. Key Findings (bulleted, color-coded by severity)

3. Detailed Analysis (each finding in a <details> block)
4. Data Tables (with sortable columns via inline JS)
5. Recommendations (numbered, prioritized)
6. Appendix: methodology + assumptions (collapsed by default)

Style:

- Max-width 1024px
- Tables with sticky header, alternate row colors
- Use color coding: red for risks, green for positives, yellow for warnings
- Dark mode support

Output: complete HTML.

典型受益：incident report、financial analysis、competitor analysis、A/B test results。

場景 3：Wiki / 知識整理

為何適合 **HTML**：知識條目要回來查。HTML 可以加 search、sticky TOC、anchor links，讓單一文件就是 mini wiki。

Prompt 範本：

Create a comprehensive wiki entry on: [主題]

Format as a complete HTML document with:

- Title + 1-paragraph TL;DR at top
- Sticky left TOC with anchor links to each section
- Search input at top that filters sections (inline JS, simple text match)
- Main sections:
 - * Definition / Overview
 - * History / Context
 - * Core Concepts (each in a <details>)
 - * Common Patterns / Examples (code blocks)
 - * Pitfalls / Anti-patterns
 - * Related Topics (links to other wiki entries you'd recommend)
 - * Further Reading (external links)
- Style: clean, Notion-like, dark mode

Output: complete HTML.

典型受益：技術概念深度整理、人物 / 公司研究、產品學習筆記、第二大腦條目。

場景 4：Dashboard / 數據儀表板

為何適合 **HTML**：dashboard 是 HTML 的原生形式。LLM 可以生成 SVG / Canvas / Chart.js 圖表，讓你拿到一個靜態但豐富的 dashboard。

Prompt 範本：

Create a dashboard for the following metrics: [paste metrics data]

Format as a complete HTML document:

- Top: 4-6 KPI cards in a grid (each: metric name / value / delta vs previous / sparkline SVG)
- Middle: 2-3 charts (bar / line / pie) using inline SVG or Chart.js via CDN
- Bottom: detailed data table with sortable columns

- Filter controls at top (date range, category)
- Dark mode by default

Use Chart.js via: `<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>`

Output: complete HTML.

典型受益：個人指標追蹤、團隊 metric review、A/B test dashboard、實驗結果展示。

場景 5：Tutorial / 教學文件

為何適合 **HTML**：教學需要分階段揭露（step-by-step）+ 可選深度（advanced 細節給願意看的人）+ 可實際嘗試（code 可複製、可執行範例）。

Prompt 範本：

Create a hands-on tutorial for: [學習主題]

Format as a complete HTML document:

- Top: Prerequisites + What you'll learn (highlighted boxes)
- Progress indicator (e.g., "Step 3 of 8") at top
- Each step in its own section:
 - * Step number + title
 - * Explanation
 - * Code block with "Copy" button (inline JS)
 - * Expected output
 - * `<details>` "Why this works" (deeper explanation, collapsed)
 - * `<details>` "Common mistakes" (collapsed)
- Final section: "Next Steps" + "Further Reading"
- Style: friendly, lots of breathing room, dark mode

Output: complete HTML.

典型受益：上手某個 library、學一個 framework、入門概念解釋、新人 onboarding 文件。

場景 6：Cheatsheet / 速查表

為何適合 **HTML**：cheatsheet 要密度高 + 可掃 + 可印。HTML 的 grid layout + 配色 + 列印優化（@media print）剛好涵蓋。

Prompt 範本：

Create a one-page cheatsheet for: [工具 / 語言 / 主題]

Format as a complete HTML document optimized for both screen and print:

- Title at top
- 2-3 column grid layout (responsive: 1 column on mobile)
- Each section is a compact card with:
 - * Section title
 - * Code snippets (small font, dense)
 - * Brief annotations
- Use color coding consistently (e.g., blue for syntax, green for examples, red for gotchas)
- @media print: A4 size, smaller fonts, no page breaks within cards
- Dark mode for screen, light mode forced for print

Output: complete HTML, optimized for density.

典型受益：常用指令速查（git / vim / docker / sql）、API reference、語法快速回顧。

場景 7：Code Review / 技術評審

為何適合 **HTML**：code review 要比較 **before/after** + 逐項評論 + 嚴重度分級。HTML 的 split view + 顏色 + 摺疊剛好對應。

Prompt 範本：

```
Review the following code: [paste code or diff]

Format as a complete HTML document:
- Top: Overall verdict (Pass / Pass with revisions / Major issues) as a colored badge
- Summary section: top 3 issues, top 3 strengths
- Detailed comments:
  * Each issue as a card with:
    - Severity badge (Critical / High / Medium / Low / Nit)
    - Code snippet (the offending lines)
    - Why it's an issue
    - Suggested fix (code block)
    - <details> "Further context" (collapsed)
- Section: "Recommended next steps"
- Style: similar to GitHub PR review, but cleaner

Output: complete HTML.
```

典型受益：自我 code review、PR 撰寫前的 self-check、架構評審、安全審計。

對照表：場景 vs HTML 元素

場景	主要 HTML 元素	為何
Plan	<details> + table + sticky TOC	階段揭露 + 結構化
Report	colored box + collapsible + tables	先看摘要再深入
Wiki	TOC + search input + anchor	重複查
Dashboard	grid + SVG / Chart.js	數據密度
Tutorial	numbered sections + code + 複製鈕	分步操作
Cheatsheet	grid layout + 列印優化	密度 + 可印
Code Review	severity badge + split view	比較 + 分級

反例：不適合 HTML 的場景

場景	為何不適合
「Python 的 lambda 是什麼？」	一句話回答，HTML 是 overkill
「翻譯這段成英文」	純文字 input/output，HTML 沒加值
「Brainstorm 10 個產品名」	列表 markdown 就夠

場景	為何不適合
即時對話	互動延遲太長，影響 flow
短回覆 / 確認類	HTML 樣板的「自重」大於內容

判準：如果輸出 < 200 字，純 markdown / 純文字就好。

觸發 HTML 的個人 SOP

建議：在你的 prompt snippet 工具（Raycast / Alfred / Espanso）設兩個快捷鍵：

快捷鍵	動作
<code>;html</code>	append Level 2 (基礎) HTML 指令
<code>;htmlpro</code>	append Level 4 (專業) HTML 指令

問問題前先評估：「這個輸出我會看超過 1 次嗎？」如果會，先按 `;htmlpro`，再 enter。

本章小結

場景	啟動條件	受益最大
Plan	多階段 + 多任務	sprint plan、deploy runbook
Report	數據 + 結論 + 細節	incident report、財報分析
Wiki	會回來查	技術概念、人物研究
Dashboard	多指標 + 圖表	個人 / 團隊追蹤
Tutorial	階段教學	上手 framework、onboarding
Cheatsheet	速查 + 列印	常用指令、API ref
Code Review	比較 + 分級	PR、架構評審

接下來

 [Chapter 04: Prompt 模板庫](#)

把 7 個場景的 prompt 集中成可複製貼上的庫。

[← 回 README](#) · [← Ch 03](#)

第 04 章：Prompt 模板庫

使用方式：直接複製對應模板，把 [...] 內容替換成你的具體輸入。9 個模板分 3 級：基礎（Level 1-2）、實戰（Level 3）、進階（Level 4 含互動）。

共用 STYLE BLOCK（基礎樣式，可重複使用）

STYLE REQUIREMENTS:

- Font family: -apple-system, BlinkMacSystemFont, "Segoe UI", "PingFang TC", sans-serif
 - Max-width: 820px (or wider for tables: 1100px)
 - Margin: 2rem auto, padding: 0 1.5rem
 - Line-height: 1.65
 - Light mode: background #fff, color #222
 - Dark mode (prefers-color-scheme): background #1a1a1a, color #ddd
 - Headings: border-bottom 1px solid, padding-bottom 0.3em, margin-top 1.8em
 - Code: font-family Menlo/Consolas/monospace, background #f5f5f5 (dark: #2a2a2a), border-radius 4px, padding 0.1em 0.35em
 - Pre: background #f5f5f5 (dark: #2a2a2a), padding 0.9em, border-radius 5px, overflow-x auto
 - Tables: full width, border-collapse collapse, alternate row backgrounds
 - Blockquote: border-left 4px solid #888, padding-left 1em, color #555 (dark: #aaa)
 - Links: color #0366d6, no underline
-

Level 1-2：基礎模板

模板 #1：通用「+HTML」

最短、最通用，無腦 append 在任何 prompt 後面。

Format your entire response as a complete HTML document with:

- Clean modern typography using system fonts
- Max-width 820px, centered
- Dark/light mode via prefers-color-scheme
- Headings with subtle border-bottom
- Code blocks with monospace font and gray background

Output only the HTML, no markdown wrapper, no explanation before/after.

模板 #2：「給我一份精美回答」

當你想要視覺驚艷時用。

Format your response as a beautifully designed HTML document.

Treat it like crafting a polished blog post or design portfolio piece:

- Generous whitespace
- Carefully chosen colors with good contrast
- Considered typography hierarchy
- Pull quotes for key insights (large italic, indented)
- Subtle animations on hover (CSS transitions)
- Dark mode that looks intentional, not just inverted

Take pride in the visual craft. Output only the HTML.

Level 3：實戰模板（7 大場景對應）

模板 #3 : Plan 規劃文件

[你的目標 / 要規劃的事情]

Generate a detailed implementation plan, formatted as a complete HTML document.

STRUCTURE:

1. Top: Goal + Success Criteria in a highlighted box
2. Executive Summary (3 sentences)
3. "Phases" section with 3-5 phases:
 - Phase 1 in <details> open by default
 - Other phases in <details> collapsed
 - Each phase contains: task table (Task / Owner / Estimate / Dependencies / Status)
4. "Risks & Mitigations" table
5. "Timeline" as ASCII Gantt or simple inline SVG
6. "Definition of Done" checklist

[paste STYLE BLOCK here]

Output: complete HTML, max-width 1100px (wider for tables).

模板 #4 : Report 分析報告

Analyze this [情境 / 數據]: [paste data or context]

Produce a comprehensive HTML report.

STRUCTURE:

1. Executive Summary box at top (3 sentences max, colored background)
2. Key Findings (3-5 bullets, color-coded: red=risk, green=positive, yellow=warning)
3. Detailed Analysis section, each finding in a <details> block:
 - The finding
 - Supporting data
 - Implications
4. Data tables with sticky headers
5. Recommendations (numbered, prioritized 1=highest)
6. Appendix in <details>: methodology, assumptions, raw data

[paste STYLE BLOCK]

Output: complete HTML, max-width 1024px.

模板 #5 : Wiki 知識整理

Create a comprehensive wiki entry on: [主題]

Format as a complete HTML document.

STRUCTURE:

- Title + 1-paragraph TL;DR
- Sticky left sidebar with TOC (anchor links to sections)
- Top: search input that filters sections by text (inline JS)
- Main sections:

- * Definition / Overview
 - * History / Context (<details> collapsed)
 - * Core Concepts (each subconcept in <details>)
 - * Common Patterns + Code Examples
 - * Pitfalls / Anti-patterns
 - * Related Topics (cross-link suggestions)
 - * Further Reading
- Each section has anchor id matching TOC

[paste STYLE BLOCK]

Output: complete HTML.

模板 #6 : Dashboard

Create a dashboard for: [metrics description or data paste]

Format as a complete HTML document.

STRUCTURE:

- Top: 4–6 KPI cards in CSS grid (each: metric name / large value / delta vs previous / inline SVG sparkline)
- Middle: 2–3 charts using inline SVG (bar chart, line chart, pie chart)
- Optional: include Chart.js via <script src="https://cdn.jsdelivr.net/npm/chart.js"></script> for complex charts
- Bottom: data table with sortable columns (inline JS)
- Filter controls at top (date range selector, category dropdown)

[paste STYLE BLOCK]

STYLE override:

- Dark mode by default (dashboards look better dark)
- KPI cards with subtle border, padding 1.5rem
- Charts max-height 300px

Output: complete HTML.

模板 #7 : Tutorial 教學

Create a hands-on tutorial for: [學習主題]

Format as a complete HTML document.

STRUCTURE:

- Top: "Prerequisites" + "What you'll learn" in side-by-side boxes
- Progress bar at top showing "Step N of M"
- Each step in its own <section>:
 - * Step number badge + title
 - * Explanation
 - * Code block with "Copy" button (inline JS)
 - * "Expected output" preview
 - * <details> "Why this works" (deeper explanation)
 - * <details> "Common mistakes"
- Final "Next Steps" + "Further Reading"

[paste STYLE BLOCK]

STYLE override:

- Friendly, lots of whitespace
- Step number badges: circle, colored
- Code blocks: monospace, syntax-aware via class (no actual highlighting unless you can do it inline)

Output: complete HTML.

模板 #8 : Cheatsheet 速查表

Create a one-page cheatsheet for: [工具 / 語言 / 框架]

Format as a complete HTML document optimized for both screen and print.

STRUCTURE:

- Title at top
- 3-column CSS grid (1 column on mobile via @media)
- Each section a compact card:
 - * Section title (small, all caps)
 - * Code snippets (dense, small font)
 - * Brief annotations
- Color coding: blue=syntax/keyword, green=examples, red=gotchas/warnings
- Dense layout, minimal whitespace

@media print:

- A4 size, no page breaks within cards
- Light mode forced
- Smaller font sizes

[paste STYLE BLOCK]

Output: complete HTML.

模板 #9 : Code Review

Review this code:

[paste code or diff]

Format your review as a complete HTML document.

STRUCTURE:

- Top: Overall verdict badge (Pass  / Pass with revisions  / Major issues )
- Summary: top 3 issues + top 3 strengths
- Detailed comments – each issue is a card containing:
 - * Severity badge (Critical=red / High=orange / Medium=yellow / Low=blue / Nit=gray)
 - * File:line reference
 - * Code snippet (the offending lines, with line numbers)
 - * "Why this is an issue" explanation
 - * "Suggested fix" with code block
 - * <details> "Further context"
- "Recommended next steps" section

[paste STYLE BLOCK]

STYLE override:

- Reuse GitHub PR-review aesthetic but cleaner
- Severity badges with appropriate colors
- Code diffs: red background for - lines, green for + lines

Output: complete HTML.

Level 4 : 進階模板 (含互動)

模板 #10 : 互動式分析儀表板 (terminal velocity 版)

Analyze the following [數據 / 主題]: [paste]

Create a self-contained interactive HTML dashboard for this analysis.

STRUCTURE:

- Hero section at top: title + 1-line TL;DR + 3 high-level KPIs
- Tabbed interface (vanilla JS tab switching) with sections:
 - Tab 1: Overview
 - Tab 2: Deep Dive
 - Tab 3: Data
 - Tab 4: Methodology
- Each tab content area has its own sub-structure
- Filters at top affect all tabs simultaneously
- Inline charts using Chart.js via CDN
- Data table on "Data" tab: sortable, searchable, exportable to CSV (inline JS)

INTERACTIVITY:

- Tab switching: click to switch, keyboard arrows to navigate
- Filters: dropdowns + date range; updating filter re-renders charts
- Table: click column header to sort, type in search box to filter
- "Copy" buttons on all code blocks
- "Export" buttons that generate CSV via Blob + download link

ACCESSIBILITY:

- Semantic <article>, <nav>, <section>
- ARIA labels
- Keyboard-navigable (Tab to focus, Enter/Space to activate)

[paste STYLE BLOCK]

Output: complete self-contained HTML (Chart.js via CDN is OK).

模板 #11 : 學習路徑生成器 (Spaced Repetition Mode)

Generate a complete learning path for: [主題]

Create an interactive HTML learning roadmap.

STRUCTURE:

- Top: Goal statement + estimated total time
- Phases as horizontal timeline (CSS flex)
- Each phase contains:
 - * Resources list (links with type badges: video / article / book / exercise)

- * Practice questions (collapsible <details> with answer reveal)
- * Self-check checklist (with checkboxes – clicking persists state in localStorage)
- Bottom: "Show progress" button that highlights which checkboxes are done

INTERACTIVITY:

- Checkbox state persists in localStorage
- "Reset progress" button
- Each checkbox marked "complete" reduces phase opacity slightly (visual progress)
- Random "Quiz me" button that picks a random question

[paste STYLE BLOCK]

Output: complete self-contained HTML.

進階組合：把模板組裝成 LLM 工作站

把這些模板集中在一個檔案 `~/.prompts/html-templates.md`，並用 Raycast / Alfred 的 snippet 功能設快捷鍵：

快捷鍵	觸發模板
<code>;html</code>	#1 通用
<code>;htmlbeauty</code>	#2 精美
<code>;htmlplan</code>	#3 Plan
<code>;htmlreport</code>	#4 Report
<code>;htmlwiki</code>	#5 Wiki
<code>;htmldash</code>	#6 Dashboard
<code>;htmltutorial</code>	#7 Tutorial
<code>;htmlcheat</code>	#8 Cheatsheet
<code>;htmlreview</code>	#9 Code Review
<code>;htmlinteractive</code>	#10 互動儀表板

問完問題後按一下快捷鍵 → send。5 秒成本，產出價值差 10 倍。

模板使用心法

心法 1：先用最小模板，再追加細節

不要一開始就用 Level 4。先用 #1 看 LLM 給什麼，再根據結果手動 **prompt** 補強：「加上 X」、「把 Y 改成 dark mode」。

心法 2：保留結果，用迭代而非從頭

LLM 第一次給的 HTML 可能 70% 達標。不要重 **prompt**，而是貼回去說：「保留所有 style 不動，但把 Section 3 改成 ...」。比較省 token，結果更穩定。

心法 3：把好的輸出存為自己的模板

LLM 給你一個你很滿意的 HTML？把它的 `<style>` 區塊抽出來存成你自己的 STYLE BLOCK。下次 prompt 直接貼這個 style → LLM 會保持風格一致。

心法 4：跨 LLM 一致性靠 STYLE BLOCK

換 LLM provider (Claude → GPT → Gemini) 會有風格漂移。把 STYLE BLOCK 越寫越具體 (顏色 hex 都明確)，漂移會縮小到可接受範圍。

接下來

[➡ Chapter 05: 反模式 — 別這樣用](#)

[← 回 README](#) · [← Ch 04](#)

第 05 章：反模式 — 別這樣用

核心句：HTML 輸出是強力工具，但用錯場合會反而降低生產力。本章列出 8 個常見陷阱，幫你判斷「這次該用 HTML 嗎？」

反模式 #1：用 HTML 包一句話回答

❌ 錯：> Q：Python enumerate 怎麼用？> A：[300 行 HTML，裡面只有「`enumerate(iterable) returns (index, value) tuples`」這 1 行真內容]

為什麼壞：HTML wrapper 自重 200 行 boilerplate，內容只 1 行。產生「樣板包雜訊」。

✅ 正解：短回答用純 markdown / plain text。HTML 是給「結構性內容」用的，不是給任何輸出都用的。

判準：如果輸出 < 200 字、或沒有 section 結構，不要 +HTML。

反模式 #2：HTML 輸出當 chat reply

❌ 錯：在 ChatGPT / Claude.ai 的 chat 介面用 +HTML，期待 chat bubble 渲染 HTML。

為什麼壞：- Chat UI sandbox HTML，多數樣式不生效 - 即使顯示，也被 chat 框架尺寸鎖住 - 沒辦法存檔、分享、用 DevTools 調

✅ 正解：HTML 輸出永遠應該存成檔案、用瀏覽器打開。Chat UI 是 review 用，不是消費用。

工作流：

反模式 #3：要求 HTML 但不指定 style

❌ 錯：「Format as HTML.」（句點，沒了）

為什麼壞：LLM 會給你極醜的 **default HTML**——白底黑字、Times New Roman、無 spacing。會比 **markdown** 還難讀。

✅ 正解：永遠 + STYLE BLOCK（見 [Ch 04](#)）。最少要指定：- 字型（system fonts）- max-width - line-height - 暗色模式

「HTML 不給樣式 = 比 markdown 還慘」。

反模式 #4：用 HTML 當「美化外殼」隱藏內容貧乏

❌ 錯：內容只有 3 個 bullet point，硬要包成 dashboard + KPI cards + charts，造成「設計感勝過資訊量」的「**deck 詐欺**」。

為什麼壞：- 讀者預期 dashboard 等級資訊，實際只有 3 點 - 浪費 token / 時間 / 注意力 - 自欺欺人（看起來像做了很多事，其實沒）

✅ 正解：內容深度先，呈現形式跟著內容走。3 個 bullet 就好好寫 3 個 bullet。

判準：「如果 LLM 給的是純文字版，內容夠豐富嗎？」如果不夠，HTML 也救不了。

反模式 #5：迭代時整份重新生成

❌ 錯：第一版 HTML 出來不滿意 → 重新 prompt 從頭來 → 結果樣式變了，連好的部分都丟了。

為什麼壞：- 浪費 token - 結果不穩定（LLM 隨機性）- 失去先前 iteration 的精華

✅ 正解：保留 + 局部修改模式。把第一版 HTML 貼回 prompt：

```
Here's the current HTML:  
[paste full HTML]
```

```
Modify ONLY:  
- Section 3: change to ...  
- The KPI on the top right: update value to ...
```

```
Keep all other CSS and structure identical. Output the complete modified HTML.
```

反模式 #6：在 HTML 裡塞「人類看不到」的 metadata

❌ 錯：要求 LLM 在 HTML 裡塞 hidden comments 給「下一個 LLM 看」（meta-prompt within HTML）。

為什麼壞：- HTML 是給人讀的，不是 agent 中間態 - 把不同抽象層攪在一起 - Debug 時無法分辨「這段是給人看的還是給 LLM 看的」

✅ 正解：HTML 是終端輸出。如果你要做 multi-step LLM workflow (agent)，用 JSON / structured output 在中間步驟，最後一步才轉成 HTML 給人類消費。

呼應 [12-factor agents Factor 4](#)：tool 是 structured output，HTML 不是 tool 是輸出層。

反模式 #7：依賴外部 CDN 但 demo 是離線環境

❌ 錯：模板裡寫 `<script src="https://cdn.jsdelivr.net/...">`，但用戶在飛機上、會議室斷網、或要把 HTML 寄給沒網路的人。

為什麼壞：HTML 看起來沒問題，實際打開圖表全是空白。

✅ 正解：兩種策略：

1. 完全 **self-contained**：禁用外部 CDN，所有圖表用 inline SVG（適合報告、寄送）
2. 明確標註：「This HTML requires internet for Chart.js. To use offline, [...）」（適合可控環境）

模板裡加：

- ALL JS and CSS must be inline (no external CDN)
 - For charts, use inline SVG (no Chart.js)
-

反模式 #8：HTML 文件越做越複雜，反而失去 LLM 加值

❌ 錯：你發現 HTML 很神奇後，要求 LLM 做越來越複雜的事——複雜動畫、複雜互動、複雜資料 binding。最後 HTML 變成「LLM 寫的 SPA」。

為什麼壞：- LLM 寫複雜 JS 一定會有 bug - 為了 debug 你花的時間 > 自己寫的時間 - 失去原本「5 秒 prompt 換 10 倍輸出」的 ROI

✅ 正解：HTML 輸出的甜蜜點是「靜態 + 少量互動」。需要的互動限制在：- 摺疊 / 展開 (<details>) - 表格 sort (30 行 inline JS) - Tab 切換 (30 行 inline JS) - Copy 按鈕 (5 行 inline JS) - localStorage 持久化簡單狀態

超過這個範圍，請用真的 **framework** (React / Vue) 跟真的工具鏈。

三個「該用真 framework 的信號」

當你出現下面任一信號，就該停止 +HTML 路線，換 React / Vue：

信號	為何要換
「LLM 寫的 JS bug 越來越多」	LLM 不擅長複雜 stateful UI
「要做 form 驗證 / 多步驟流程」	React Hook Form / Vue 等成熟方案完爆 LLM 寫的
「要連 API、要持久化、要登入」	那是 web app 不是 HTML 文件

HTML 輸出的定位：「文件 / 報告 / 知識 / **dashboard** 等以閱讀為主的東西」。Web app 不在範圍內。

反模式比較表

反模式	信號	解法
#1 包一句話	內容 < 200 字	用 markdown
#2 chat 介面渲染	HTML 顯示異常	存檔用瀏覽器開
#3 不指定 style	預設 ugly HTML	加 STYLE BLOCK
#4 美化包貧乏	設計遠勝內容	加深內容
#5 整份重生成	樣式隨機飄	保留 + 局部改
#6 hidden metadata	LLM-to-LLM 訊息混在 HTML	分離抽象層
#7 線上 CDN 依賴	離線打開壞掉	self-contained
#8 LLM 寫 SPA	JS bug 多	換真 framework

自我檢查清單

下次要用 +HTML 之前，問自己這 5 題：

- 1. ☐ 這個輸出我會看超過 1 次嗎？
- 2. ☐ 這個輸出有 ≥ 3 個 sections？
- 3. ☐ 我有 STYLE BLOCK 嗎？
- 4. ☐ 我要不要 self-contained（無 CDN）？
- 5. ☐ 我預期是「文件」還是「互動 web app」？（後者請換 framework）

5 題都對的，下手 +HTML。否則用更輕量的方案。

接下來

[➡ Chapter 06: 進階 — 互動、可下載、分享](#)

在「靜態文件」的天花板內，怎麼把 HTML 輸出做到極致。

第 06 章：進階 — 互動、可下載、分享

核心句：靜態 HTML 的天花板比你想像的高。在「不寫 **build step**、不引外部 **framework**」的前提下，你可以加上摺疊、tab、sort、search、copy、localStorage 持久化、PDF 列印優化、可分享 link，把 HTML 輸出從「文件」升級成「mini app」。

進階範圍的明確邊界

本章談的是「**inline JS** 不超過 100 行能實作」的互動。超過這個，請見 [反模式 #8](#)，換真 framework。

6 個 inline JS 互動 pattern

Pattern 1：可摺疊區塊（純 HTML，零 JS）

最便宜的互動，不用任何 JS：

```
<details>
  <summary>展開：詳細推導</summary>
  <p>因為 X，所以 Y...</p>
</details>
```

加上樣式：

```
details {
  border: 1px solid #ddd;
  border-radius: 4px;
  padding: 0.5em 1em;
  margin: 0.5em 0;
}
summary {
  cursor: pointer;
  font-weight: 600;
}
details[open] summary {
  margin-bottom: 0.5em;
}
```

Prompt 加：

Use <details>/<summary> for any content the reader might want to skip.
Default state: open for primary sections, collapsed for secondary.

Pattern 2：Tab 切換（30 行 JS）

```
<div class="tabs">
  <nav>
    <button data-tab="overview" class="active">Overview</button>
    <button data-tab="details">Details</button>
  </nav>
  <div>
    <div data-tab="overview">Overview content</div>
    <div data-tab="details">Details content</div>
  </div>
</div>
```



```

    <button data-tab="data">Data</button>
  </nav>
  <section data-content="overview" class="active">...</section>
  <section data-content="details" hidden>...</section>
  <section data-content="data" hidden>...</section>
</div>

<script>
document.querySelectorAll('.tabs nav button').forEach(btn => {
  btn.addEventListener('click', () => {
    const target = btn.dataset.tab;
    btn.parentElement.querySelectorAll('button').forEach(b =>
      b.classList.toggle('active', b === btn));
    btn.closest('.tabs').querySelectorAll('section').forEach(s => {
      const isTarget = s.dataset.content === target;
      s.classList.toggle('active', isTarget);
      s.hidden = !isTarget;
    });
  });
});
</script>

```

Pattern 3 : 表格 sort (40 行 JS)

```

<table id="data-table">
  <thead>
    <tr>
      <th data-sort="name">Name ▲▼</th>
      <th data-sort="value">Value ▲▼</th>
      <th data-sort="date">Date ▲▼</th>
    </tr>
  </thead>
  <tbody>...</tbody>
</table>

<script>
const tbl = document.getElementById('data-table');
const tbody = tbl.querySelector('tbody');
let sortState = {col: null, asc: true};

tbl.querySelectorAll('th[data-sort]').forEach((th, i) => {
  th.style.cursor = 'pointer';
  th.addEventListener('click', () => {
    const col = th.dataset.sort;
    sortState.asc = sortState.col === col ? !sortState.asc : true;
    sortState.col = col;
    const rows = Array.from(tbody.querySelectorAll('tr'));
    rows.sort((a, b) => {
      const av = a.children[i].textContent.trim();
      const bv = b.children[i].textContent.trim();
      const aNum = parseFloat(av), bNum = parseFloat(bv);
      const cmp = !isNaN(aNum) && !isNaN(bNum)
        ? aNum - bNum
        : av.localeCompare(bv);
      return sortState.asc ? cmp : -cmp;
    });
    rows.forEach(r => tbody.appendChild(r));
  });
});

```

```
});  
</script>
```

Pattern 4 : Search filter (20 行 JS)

```
<input id="search" placeholder="Filter...">  
<table>  
  <tbody id="searchable">  
    <tr><td>foo</td><td>123</td></tr>  
    <tr><td>bar</td><td>456</td></tr>  
  </tbody>  
</table>  
  
<script>  
document.getElementById('search').addEventListener('input', (e) => {  
  const q = e.target.value.toLowerCase();  
  document.querySelectorAll('#searchable tr').forEach(tr => {  
    tr.hidden = !tr.textContent.toLowerCase().includes(q);  
  });  
});  
</script>
```

Pattern 5 : Copy 按鈕 (15 行 JS)

```
<pre><code>fetch_data --output csv</code><button class="copy-btn">Copy</button></pre>  
  
<script>  
document.querySelectorAll('.copy-btn').forEach(btn => {  
  btn.addEventListener('click', () => {  
    const code = btn.previousElementSibling.textContent;  
    navigator.clipboard.writeText(code);  
    const orig = btn.textContent;  
    btn.textContent = 'Copied!';  
    setTimeout(() => btn.textContent = orig, 1500);  
  });  
});  
</script>
```

Pattern 6 : localStorage 持久化 (25 行 JS)

讓用戶的 checkbox 狀態 / 折疊狀態跨 session 保留：

```
<label><input type="checkbox" data-persist="task-1"> 完成設計 Spec</label>  
<label><input type="checkbox" data-persist="task-2"> 完成 API 文件</label>  
  
<script>  
document.querySelectorAll('input[data-persist]').forEach(input => {  
  const key = 'persist:' + input.dataset.persist;  
  input.checked = localStorage.getItem(key) === '1';  
  input.addEventListener('change', () => {  
    localStorage.setItem(key, input.checked ? '1' : '0');  
  });  
});  
</script>
```

PDF 列印優化

當你要把 HTML 印成 PDF（Chrome Cmd+P → “Save as PDF”），加這段 CSS：

```
@media print {
  body {
    max-width: none !important;
    margin: 0 !important;
    background: white !important;
    color: black !important;
  }

  /* 避免在 card / table 中間斷頁 */
  .card, table, pre {
    break-inside: avoid;
  }

  /* 隱藏互動元素 */
  .copy-btn, .search-input, nav.tabs {
    display: none !important;
  }

  /* 強制展開所有 details */
  details {
    break-inside: avoid;
  }
  details[open] summary {
    margin-bottom: 0;
  }

  /* 連結後面顯示 URL */
  a[href]:after {
    content: " (" attr(href) " ";
    font-size: 0.85em;
    color: #666;
  }

  /* A4 紙設定 */
  @page {
    size: A4;
    margin: 1.5cm;
  }
}
```

Prompt 加：

Include @media print CSS for:

- Removing max-width and dark colors
- Avoiding break-inside on cards/tables/pre
- Hiding interactive UI (.copy-btn, search input)
- Force-expanding all <details>
- A4 page size with 1.5cm margins

可下載成檔案

讓 HTML 自己生成 CSV / JSON / Markdown 給用戶下載：

```
<button id="export-csv">Export as CSV</button>

<script>
document.getElementById('export-csv').addEventListener('click', () => {
  const rows = [['Name', 'Value']];
  document.querySelectorAll('#data-table tbody tr').forEach(tr => {
    rows.push(Array.from(tr.children).map(td => td.textContent.trim()));
  });
  const csv = rows.map(r => r.map(c => `${c.replace(/"/g, '""')}").join(',')').join('\n');
  const blob = new Blob([csv], {type: 'text/csv'});
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'data.csv';
  a.click();
  URL.revokeObjectURL(url);
});
</script>
```

可分享：把 HTML 當 single file 分享

HTML 的最大優勢之一是「一個檔案就是完整作品」，可以：

1. **Email** 附件：HTML 直接收件人雙擊就能看
2. **AirDrop** / 微信檔案：同樣
3. 存 **iCloud** / **Dropbox** / 公司 **wiki**：當 first-class document
4. 存 **git**：版控
5. 用 [GitHub Gist](#) 公開：得到一個可分享 link

進階：把 HTML 轉成 **data URL** 直接做成超連結：

```
# macOS: 把 HTML 轉 data URL，複製到剪貼簿
DATA_URL="data:text/html;base64,$(base64 -i response.html)"
echo "$DATA_URL" | pbcopy
# 貼到瀏覽器網址列直接打開（沒檔案傳遞需要）
```

注意 data URL 有長度限制（Chrome ~ 2MB），太大檔案不適合。

把 HTML 變成 PWA（offline-first）

加 `<link rel="manifest">` + service worker，理論上 HTML 可以 install 到桌面當 app。但這超出本章範圍，需要 build step。如果你有這需求，請用真的 **PWA** 工具鏈。

進階組合：個人「LLM Document Pipeline」

完整工作流範例：

```
#!/usr/bin/env python3
# ~/bin/llm-html
import sys, os, subprocess, anthropic, datetime

TEMPLATES = {
    'report': open('~/.prompts/html-report.md').read(),
    'plan': open('~/.prompts/html-plan.md').read(),
    'wiki': open('~/.prompts/html-wiki.md').read(),
    # ...
}

def main():
    template_name = sys.argv[1] # 'report' / 'plan' / 'wiki'
    user_prompt = sys.stdin.read()

    client = anthropic.Anthropic()
    full_prompt = f"{user_prompt}\n\n---\n\n{TEMPLATES[template_name]}"

    msg = client.messages.create(
        model="claude-opus-4-7",
        max_tokens=16000,
        messages=[{"role": "user", "content": full_prompt}]
    )

    html = msg.content[0].text.strip()
    # 剝除可能的 markdown wrapper
    if html.startswith('```html'):
        html = html[7:]
    if html.endswith('```'):
        html = html[:-3]

    # 存到 ~/Documents/llm-outputs/
    out_dir = os.path.expanduser('~/.Documents/llm-outputs')
    os.makedirs(out_dir, exist_ok=True)
    ts = datetime.datetime.now().strftime('%Y%m%d-%H%M%S')
    path = f'{out_dir}/{template_name}-{ts}.html'

    with open(path, 'w') as f:
        f.write(html.strip())

    # 自動打開
    subprocess.run(['open', path])
    print(f'✓ Saved: {path}')

if __name__ == '__main__':
    main()
```

使用：

```
echo "Analyze my Q1 sales data: ..." | llm-html report
echo "Plan migration to PostgreSQL" | llm-html plan
echo "Wiki entry on Kubernetes Operators" | llm-html wiki
```

進階心法

心法 1：JS 互動 < 100 行就是甜蜜點

超過 100 行 inline JS，debug 成本 > 自己寫的成本。守住這條線。

心法 2：CSS 投資 > JS 投資

漂亮的 CSS（顏色、間距、字型、轉場）給讀者的價值，遠大於同樣 token 的 JS 互動。先投資 CSS，再考慮 JS。

心法 3：每個互動都要在沒 JS 時也能用

漸進增強原則：表格沒 JS 也能讀（只是不能 sort）、details 沒 JS 也能展開、search 沒 JS 至少不會 break 頁面。

心法 4：用 Chrome DevTools 反向學習

LLM 給的 HTML 你覺得很美，但你不知道怎麼來的？用 DevTools 看 source、看 computed style，把好东西抄進你的 STYLE BLOCK。

接下來

[➡ Chapter 07: 跟 12-factor agents 的關聯](#)

[← 回 README](#) · [← Ch 06](#)

第 07 章：跟 12-Factor Agents 的關聯

核心句：Karpathy 的「+HTML」跟 Dex Horthy 的「12-factor agents」乍看是兩個獨立社群提的不同方法。實際上它們同源——都在解決「LLM 內部能力遠超 chat UI 表達」的不對稱問題。把這兩個觀念放在一起，會理解整套 LLM-as-software 的設計哲學。

兩條觀念的根

觀念	提出者	核心句	解決什麼
Structure as HTML	Karpathy	「Output is still stuck in the Stone Age」	LLM 輸出層還停在純文字
12-Factor Agents	Dex Horthy	「Agent 本質上是穿插 LLM 步驟的 deterministic 軟體」	LLM 工程化還停在「丟 prompt loop until done」

不同層，但同源——LLM 的潛能被「過度簡化的使用模式」鎖住了。

對應關係

Karpathy +HTML ↔ Factor 3 (Context Engineering)

Factor 3 : **You don’t necessarily need to use standard message-based formats for conveying context to an LLM.**

把這句反過來：你也不必用標準 **message format** 接收 LLM 輸出。

維度	Factor 3 (input side)	Karpathy +HTML (output side)
拒絕的預設	“Standard ChatML messages”	“Plain text / markdown response”
推薦做法	自製 XML-like context format	自製 HTML 輸出格式
為什麼	Token efficiency + attention	Reading efficiency + reuse

兩條原則的精神完全一致：**Take ownership of the format**——不要被 chat UI / framework 默認鎖死。

Karpathy +HTML ↔ Factor 4 (Tools are Structured Outputs)

Factor 4 : **Tools don’t need to be complex. At their core, they’re just structured output from your LLM.**

HTML 是 **structured output** 的另一種形式——它不是給程式 parse 的，是給瀏覽器 render 的，但本質都是「LLM 輸出結構化資料」。

形式	給誰看	解析方式
JSON tool call	程式碼	JSON.parse
HTML document	瀏覽器	DOM parser
Markdown	人類	眼睛

JSON 是 machine-readable structured；HTML 是 **human-readable structured**；markdown 是 **structured-light**。+HTML 把 LLM 從「給人看的非結構化」推到「給人看的結構化」。

Karpathy +HTML ↔ Factor 2 (Own Your Prompts)

Factor 2 : **Don’t outsource your prompt engineering to a framework.**

+HTML 是 prompt engineering 的具體技藝。要把它做好，你需要：

- 自己寫 STYLE BLOCK
- 自己迭代模板
- 自己測試跨 LLM provider 的一致性
- 自己版控

完全符合 **Factor 2** 的「prompt 是 first-class code」。

Karpathy +HTML ↔ Factor 10 (Small Focused Agents)

Factor 10 : **Small, focused agents do one thing well.**

當你用 agent 自動化 HTML 生成 pipeline (例如 [Ch 06 的範例](#))，每個 template 對應一個 micro-agent：

- report-html-agent: 5-10 step，只負責生成 report
- plan-html-agent: 5-10 step，只負責生成 plan
- wiki-html-agent: 5-10 step，只負責生成 wiki

不要做一個「萬能 **HTML agent**」，做多個 focused agent。

整合架構：HTML 是 12-factor agent 的「人類消費介面」

把 12-factor 完整架構畫出來：

```
[trigger] (Factor 11)
  ↓
[agent loop] (Factor 8, 12)
  ↓
[deterministic tool execution] (Factor 4)
  ↓
[event accumulated in thread] (Factor 3, 5)
  ↓
[possibly: human in the loop] (Factor 7, 6)
  ↓
[final state: thread complete]
  ↓
[★ render thread as HTML report for human review ★] ← Karpathy +HTML 在這裡進入
```

最後一步 — 把 agent 跑完的 thread 渲染成 HTML 給人類消費 — 完全是 **Karpathy** 方法論的領域。

實戰範例：deploybot + HTML report

對應 [12-factor 第 0 章的 deploybot 範例](#)：

```
Agent 跑完一次 deploy (15 個 events 在 thread 裡)
  ↓
Deterministic code 把 thread 餵給 LLM with HTML template:
  ↓
"Generate a deployment summary as a complete HTML document with:
- Top: Deploy verdict (success / partial / failed) badge
- Timeline: chronological list of all events with timestamps
- Each tool call expandable for full input/output
- Each human approval highlighted
- Bottom: Lessons learned / what to improve next time"
  ↓
存到 reports/deployments/2026-05-19-prod-deploy.html
```


↓
Slack 通知 link 給 SRE team

Agent 自己跑完，自己生成可分享的 **HTML** 報告。這就是 12-factor agent + Karpathy +HTML 的合體應用。

工具鏈整合

把 Karpathy +HTML 寫進你的 12-factor agent codebase：

```
# agent_renderer.py

HTML_REPORT_TEMPLATE = """
Generate an HTML report summarizing this agent run.

CONTEXT:
{thread_yaml}

STRUCTURE: [Level 3 spec]
STYLE: [STYLE BLOCK]
OUTPUT: complete HTML.
"""

def render_thread_as_html(thread: Thread) -> str:
    """
    Render an agent thread (Factor 5) as a human-readable HTML report.
    Uses Karpathy's +HTML pattern at the final step.
    """
    prompt = HTML_REPORT_TEMPLATE.format(
        thread_yaml=thread_to_yaml(thread) # Factor 3: 你自己的 serialization
    )
    response = llm.complete(prompt, max_tokens=16000)
    return strip_markdown_wrapper(response)
```

render_thread_as_html() 在 **agent loop** 完成時呼叫一次，把跑了 30 分鐘的 thread 變成 1 分鐘可讀的 HTML 報告。

「Output Engineering」：新的子領域

過去業界談 prompt engineering / context engineering。Karpathy 的 +HTML 是「**Output Engineering**」這個新子領域的開端。它的問題不是「LLM 怎麼想」，而是「LLM 想完之後怎麼呈現給人類消費」。

對應的工程實踐還包括：- HTML 之外的輸出格式（Notion / Markdown enhanced / SVG / PDF / video） - 多格式同步生成（一次 LLM call 出 markdown + HTML + PDF） - Style guide 系統化（公司級別的「LLM 輸出 style」一致性） - 輸出 caching（同樣內容不要每次都 re-render）

這些都是未來 1-2 年 LLM ops 的成長領域。

對你的綜合建議

1. 內部 **agent** / **pipeline** 終端輸出：永遠加 +HTML
 2. 跟人類互動的最後一公里：用 HTML 而非 chat message
 3. 跨 **team** 分享 LLM 結果：HTML 比 markdown 更專業
 4. 公司級別的 LLM 應用：把 HTML template 庫當公司資產維護
-

兩條觀念的終極合體

觀念	一句話
12-factor agents	把 agent 當工程化軟體寫
Karpathy +HTML	把 LLM 輸出當文件作品做
合體	把 LLM 應用當「軟體 + 文件出版」雙系統設計

當你把這兩條都做好，你的 LLM 應用同時擁有：

- 工程紀律（agent 可控、可暫停、可恢復、可審計）
- 產品質感（最終輸出可讀、可分享、可印、可保存）

這是 production-grade LLM 應用的雙翼。

接下來

 [Appendix: HTML/CSS 速查](#)

[← 回 README](#) · [← Ch 07](#)

Appendix：HTML / CSS 速查（給 +HTML 工作流用）

用途：當你想精細控制 LLM 生成的 HTML 樣式時，這份是即查即用的 pattern 庫。不教你 HTML/CSS 基礎，假設你已會基本語法。

1. HTML5 boilerplate（最小可用）

```
<!doctype html>
<html lang="zh-Hant">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>標題</title>
  <style>
    /* CSS 在這裡 */
```

```
</style>
</head>
<body>
  <!-- 內容 -->
  <script>
    // JS 在這裡 (如有)
  </script>
</body>
</html>
```

2. 系統字型棧（跨平台一致）

```
font-family:
  -apple-system, BlinkMacSystemFont,    /* macOS / iOS */
  "Segoe UI",                            /* Windows */
  Roboto,                                /* Android */
  "PingFang TC", "PingFang SC",          /* macOS 中文 */
  "Microsoft JhengHei", "Microsoft YaHei", /* Windows 中文 */
  "Helvetica Neue", Arial,
  sans-serif;
```

Monospace（程式碼用）：

```
font-family:
  ui-monospace,
  Menlo, Monaco,
  "Cascadia Mono",
  "Roboto Mono",
  monospace;
```

3. 暗色模式（auto 切換）

```
:root {
  --bg: #fff;
  --fg: #222;
  --muted: #666;
  --code-bg: #f5f5f5;
  --border: #ddd;
  --accent: #0366d6;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #1a1a1a;
    --fg: #ddd;
    --muted: #999;
    --code-bg: #2a2a2a;
    --border: #444;
    --accent: #58a6ff;
  }
}

body {
  background: var(--bg);
```

```
color: var(--fg);
}
```

4. 文件最佳閱讀樣式

```
body {
  max-width: 820px;
  margin: 2rem auto;
  padding: 0 1.5rem;
  line-height: 1.65;
}

h1, h2, h3, h4 {
  border-bottom: 1px solid var(--border);
  padding-bottom: 0.3em;
  margin-top: 1.8em;
}

h1 { font-size: 1.9em; }
h2 { font-size: 1.5em; }
h3 { font-size: 1.2em; }

p { margin: 1em 0; }
```

5. 強調 box 樣式（結論 / 警告 / 提示）

```
.callout {
  border-left: 4px solid;
  padding: 1em 1.5em;
  margin: 1em 0;
  border-radius: 0 4px 4px 0;
}

.callout.info { border-color: #0366d6; background: #f1f8ff; }
.callout.success { border-color: #28a745; background: #f0fff4; }
.callout.warning { border-color: #ffd33d; background: #ffffbd; }
.callout.danger { border-color: #d73a49; background: #ffeef0; }

@media (prefers-color-scheme: dark) {
  .callout.info { background: #0a2540; }
  .callout.success { background: #0a3320; }
  .callout.warning { background: #332b00; }
  .callout.danger { background: #3a0f14; }
}
```

用法：

```
<div class="callout info">
  <strong>提示：</strong>...
</div>
```

6. 表格樣式

```
table {
  border-collapse: collapse;
  width: 100%;
  margin: 1em 0;
  font-size: 0.95em;
}

th, td {
  border: 1px solid var(--border);
  padding: 0.5em 0.8em;
  text-align: left;
  vertical-align: top;
}

thead {
  background: var(--code-bg);
  position: sticky;
  top: 0;
}

tbody tr:nth-child(even) {
  background: rgba(0, 0, 0, 0.02);
}

@media (prefers-color-scheme: dark) {
  tbody tr:nth-child(even) {
    background: rgba(255, 255, 255, 0.03);
  }
}
```

7. Code 與 Pre 樣式

```
code {
  font-family: ui-monospace, Menlo, monospace;
  background: var(--code-bg);
  padding: 0.1em 0.4em;
  border-radius: 3px;
  font-size: 0.92em;
}

pre {
  background: var(--code-bg);
  padding: 1em;
  border-radius: 6px;
  overflow-x: auto;
  font-size: 0.88em;
  line-height: 1.5;
  border: 1px solid var(--border);
}

pre code {
  background: transparent;
  padding: 0;
  font-size: inherit;
}
```

8. Details / Summary 樣式

```
details {  
  border: 1px solid var(--border);  
  border-radius: 4px;  
  padding: 0.5em 1em;  
  margin: 0.7em 0;  
}  
  
summary {  
  cursor: pointer;  
  font-weight: 600;  
  padding: 0.3em 0;  
  list-style: none;  
}  
  
summary::before {  
  content: '▶';  
  display: inline-block;  
  margin-right: 0.5em;  
  transition: transform 0.2s;  
  font-size: 0.8em;  
}  
  
details[open] summary::before {  
  transform: rotate(90deg);  
}  
  
details[open] summary {  
  border-bottom: 1px solid var(--border);  
  margin-bottom: 0.5em;  
}
```

9. KPI Card grid

```
.kpi-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));  
  gap: 1rem;  
  margin: 1.5rem 0;  
}  
  
.kpi {  
  border: 1px solid var(--border);  
  border-radius: 6px;  
  padding: 1rem 1.25rem;  
  background: var(--bg);  
}  
  
.kpi-label {  
  font-size: 0.85em;  
  color: var(--muted);  
  text-transform: uppercase;  
  letter-spacing: 0.05em;  
}  
  
.kpi-value {
```

```
font-size: 2em;
font-weight: 700;
margin: 0.3em 0;
}

.kpi-delta {
font-size: 0.9em;
font-weight: 600;
}

.kpi-delta.up { color: #28a745; }
.kpi-delta.down { color: #d73a49; }
```

10. Sticky TOC sidebar

```
.layout {
display: grid;
grid-template-columns: 220px 1fr;
gap: 2rem;
max-width: 1100px;
margin: 2rem auto;
}

.toc {
position: sticky;
top: 1rem;
align-self: start;
max-height: calc(100vh - 2rem);
overflow-y: auto;
font-size: 0.9em;
}

.toc a {
display: block;
padding: 0.3em 0;
color: var(--fg);
text-decoration: none;
border-left: 2px solid transparent;
padding-left: 0.8em;
}

.toc a.active {
border-left-color: var(--accent);
font-weight: 600;
}

@media (max-width: 768px) {
.layout { grid-template-columns: 1fr; }
.toc { position: static; max-height: none; }
}
```

11. 列印優化 (@media print)

```
@media print {
body {
max-width: none;
}
```

```

    margin: 0;
    background: white !important;
    color: black !important;
}

.toc, .copy-btn, .search, nav.tabs {
    display: none !important;
}

details {
    break-inside: avoid;
}

details summary {
    display: none;
}

details > *:not(summary) {
    display: block !important;
}

table, pre, .kpi {
    break-inside: avoid;
}

a[href]::after {
    content: " (" attr(href) ")";
    font-size: 0.85em;
    color: #666;
}

@page {
    size: A4;
    margin: 1.5cm;
}
}

```

12. 響應式佈局

```

/* mobile-first */
.layout {
    display: grid;
    grid-template-columns: 1fr;
    gap: 1rem;
}

/* tablet */
@media (min-width: 768px) {
    .layout {
        grid-template-columns: 200px 1fr;
    }
}

/* desktop */
@media (min-width: 1200px) {
    .layout {
        grid-template-columns: 220px 1fr 200px;
    }
}

```



```
}  
}
```

13. Inline SVG charts

簡單 bar chart：

```
<svg viewBox="0 0 300 150" width="100%" height="150">  
  <rect x="20" y="80" width="40" height="60" fill="#0366d6" />  
  <rect x="80" y="50" width="40" height="90" fill="#0366d6" />  
  <rect x="140" y="30" width="40" height="110" fill="#0366d6" />  
  <rect x="200" y="60" width="40" height="80" fill="#0366d6" />  
  <text x="40" y="148" text-anchor="middle" font-size="10">Jan</text>  
  <text x="100" y="148" text-anchor="middle" font-size="10">Feb</text>  
  <text x="160" y="148" text-anchor="middle" font-size="10">Mar</text>  
  <text x="220" y="148" text-anchor="middle" font-size="10">Apr</text>  
</svg>
```

簡單 sparkline：

```
<svg viewBox="0 0 100 30" width="100%" height="30">  
  <polyline points="0,20 20,10 40,15 60,5 80,8 100,3" />  
  fill="none" stroke="#0366d6" stroke-width="2" />  
</svg>
```

14. 全套 STYLE BLOCK (複製貼上即可用)

```
<style>  
:root {  
  --bg: #fff; --fg: #222; --muted: #666;  
  --code-bg: #f5f5f5; --border: #ddd; --accent: #0366d6;  
}  
@media (prefers-color-scheme: dark) {  
  :root { --bg: #1a1a1a; --fg: #ddd; --muted: #999; --code-bg: #2a2a2a; --border: #444; --  
    accent: #58a6ff; }  
}  
body {  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", "PingFang TC", "PingFang SC",  
    "Microsoft JhengHei", sans-serif;  
  background: var(--bg); color: var(--fg);  
  max-width: 820px; margin: 2rem auto; padding: 0 1.5rem;  
  line-height: 1.65;  
}  
h1, h2, h3, h4 { border-bottom: 1px solid var(--border); padding-bottom: 0.3em; margin-top:  
  1.8em; }  
h1 { font-size: 1.9em; } h2 { font-size: 1.5em; } h3 { font-size: 1.2em; }  
code { font-family: ui-monospace, Menlo, monospace; background: var(--code-bg); padding:  
  0.1em 0.4em; border-radius: 3px; font-size: 0.92em; }  
pre { background: var(--code-bg); padding: 1em; border-radius: 6px; overflow-x: auto; font-  
  size: 0.88em; line-height: 1.5; border: 1px solid var(--border); }  
pre code { background: transparent; padding: 0; }  
table { border-collapse: collapse; width: 100%; margin: 1em 0; }  
th, td { border: 1px solid var(--border); padding: 0.5em 0.8em; text-align: left; vertical-  
  align: top; }
```

```

th { background: var(--code-bg); }
tbody tr:nth-child(even) { background: rgba(0, 0, 0, 0.02); }
@media (prefers-color-scheme: dark) { tbody tr:nth-child(even) { background: rgba(255, 255, 255, 0.03); } }
details { border: 1px solid var(--border); border-radius: 4px; padding: 0.5em 1em; margin: 0.7em 0; }
summary { cursor: pointer; font-weight: 600; }
blockquote { border-left: 4px solid var(--muted); padding-left: 1em; color: var(--muted); margin: 1em 0; }
a { color: var(--accent); text-decoration: none; }
a:hover { text-decoration: underline; }
.callout { border-left: 4px solid; padding: 1em 1.5em; margin: 1em 0; border-radius: 0 4px 4px 0; }
.callout.info { border-color: #0366d6; background: rgba(3, 102, 214, 0.08); }
.callout.success { border-color: #28a745; background: rgba(40, 167, 69, 0.08); }
.callout.warning { border-color: #ffd33d; background: rgba(255, 211, 61, 0.12); }
.callout.danger { border-color: #d73a49; background: rgba(215, 58, 73, 0.08); }
@media print {
  body { max-width: none; margin: 0; background: white !important; color: black !important; }
  details summary { display: none; } details > *:not(summary) { display: block !important; }
  @page { size: A4; margin: 1.5cm; }
}
</style>

```

這份 STYLE BLOCK 可以直接貼到所有 [Ch 04 模板](#) 的 STYLE 區域，立即得到 production-grade 樣式。

結尾

恭喜你讀完整份 Karpathy +HTML 精解。

一句話帶走

任何 LLM prompt 結尾，加一句「**Format your entire response as a complete HTML document**」。

剩下的細節（樣式 / 模板 / 互動 / 反模式）都是優化。先做這一件事，就贏了 **90%** 沒做的人。

[← 回 README](#)